Team:

YuKun Cai, Jennifer Greene, James Pazik, Ryan Shaughnessy, Sarah Thompson

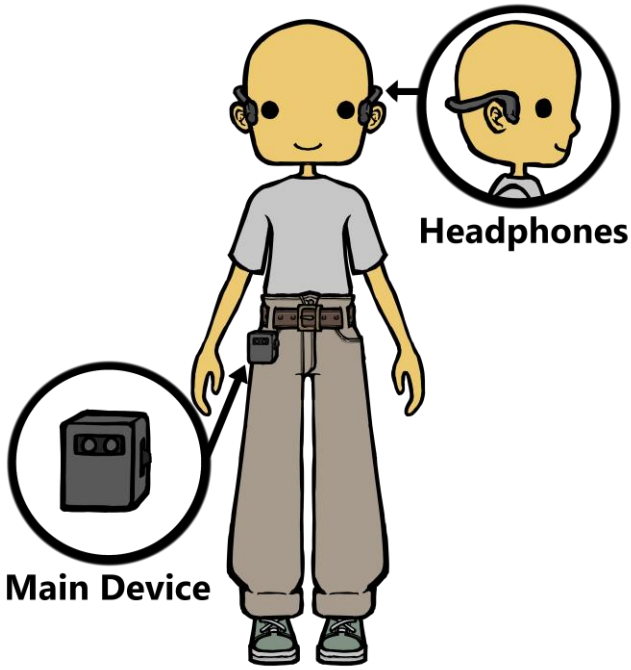
Project Title:

Third Eye Navigation

Design

I. **Brief Introduction (ST)**
   a. Third Eye Navigation is a wearable device intended to be used by people who are blind or otherwise visually impaired. It provides assistance while navigating by detecting objects within immediate walking-range of the user and alerting them accordingly when there is an obstacle in their path.
   b. The main goal of Third Eye is to make navigation safer and easier for those who are blind or visually impaired. The device will act as a third eye, seeing potential obstacles that the user may not be able to see or feel with a traditional mobility cane. By alerting the user to these obstacles, the device will help the user stay aware of their surroundings.
   c. Github Link:
      i. https://github.com/jgre138/CSSE-2026-27-senior-project

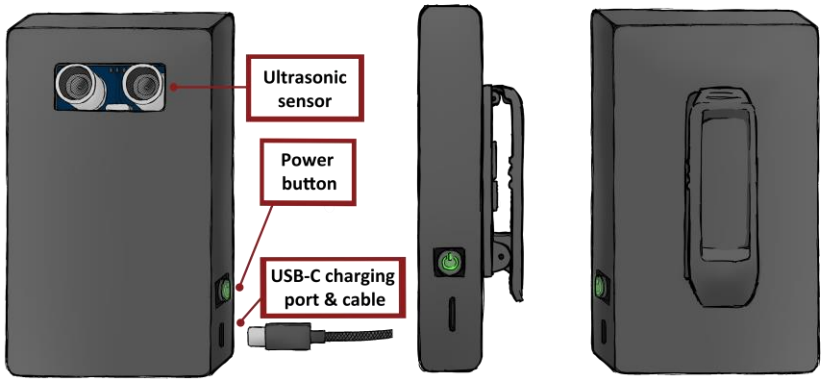
II. **Design (ST)**
   a. *User Model*



      i. Components: Main device, (OPTIONAL) Bluetooth headphones

ii. The main device will contain all hardware enclosed in a plastic case that can be clipped to the user's clothing.
iii. Bluetooth headphones will be optional; users who choose to connect their headphones will be able to receive additional directional cues.

b. Product Model



i. The case will be 3D-printed to perfectly fit all hardware components inside.
ii. There will be an opening in the front of the case for the ultrasonic sensor to be able to function; it cannot be obstructed.
iii. There will be a USB-C charging port and a power button.
iv. There will be a belt-clip on the back.

III. Algorithms and Core Logic (ST)

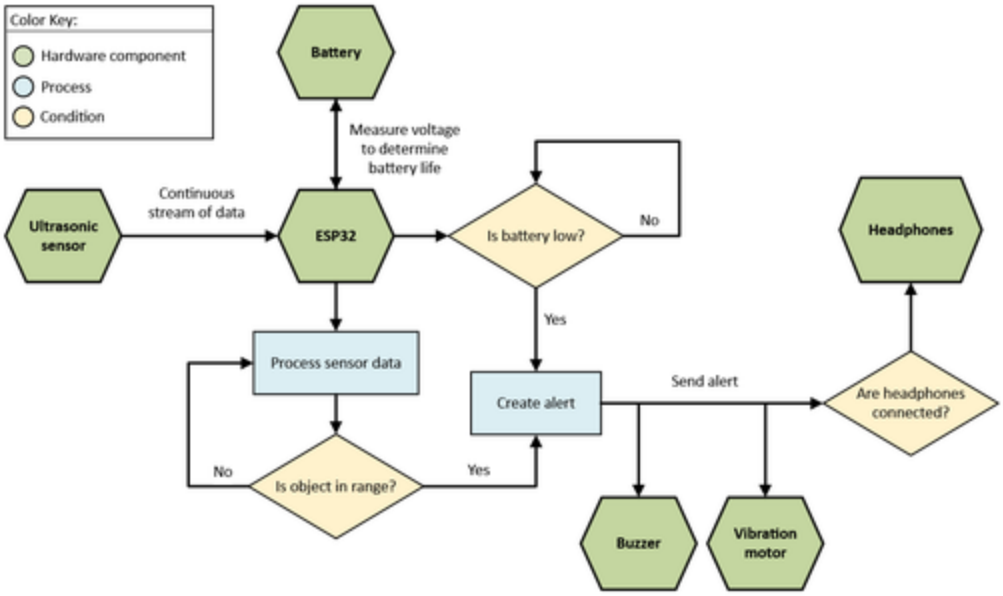
a. Alert System

i. The System will have 3 different alerts:

	Alert Trigger	Beeper/Vibration Motor Feedback	Message Feedback
1	Object detected in range of user	Beep/vibrate twice	"Upcoming obstacle on the ground/at eye-level/etc."
2	Battery indication	Beep/vibrate rapidly for a few seconds. Repeat periodically until plugged in to charger	"Battery low"
3	Power-on indication	Beep/vibrate once when device is powered on	"Power on"

ii. More alerts may be added if they are needed.

b. Alert System Flow Chart



i. The Ultrasonic sensor gives sensor data to the ESP32, which will then use logic to determine whether the object is in range of not. If so, it will create an alert according to the specifications in the table above and send a signal to the buzzer and the vibration motor for direct feedback to the user.

ii. If the optional headphones are connected, then it will send a signal to the headphones using Bluetooth connection.

iii. The ESP32 will keep tract of if the battery is low and alert if it needs to be recharged.

IV. System Architecture and Dependencies (RT & ST)

a. Component List:

Component	Purpose
ESP32 Microcontroller	Process sensor data & control other modules
Ultrasonic Sensor	Object detection
Buzzer	Audio feedback (beeping)
Vibration motor	Tactile feedback (vibration)
Charging board	To charge the device battery
Rechargeable battery	Power supply
Breadboard	To connect modules to ESP32

b. Arduino/ESP32

i. Arduino is an open source platform for electronic projects that we're using for our project. It's a circuit board with microcontrollers and pins.

ii. The microcontroller, ESP32, is the 'brain' of the machine. It contains the main software we need to run our project.
iii. All the other components will interact with this through the connection of the breadboard and the jumper wires. Powered by a battery.

c. *Breadboard/Jumper wires*
   i. The breadboard is what we're attaching all this to. It is a large flat board with many holes in it in which we will put all our hardware on. Breadboards come in many sizes, we're using a big one to demonstrate the components and for testing, we will switch to a smaller one when we move on to our final prototype.
   ii. Jumper wires allow commands to transfer from component to component. It directs the electronic pulses from point A to point B in order to do what we want.

d. *Battery*
   i. This will supply power to our circuit. We use this to have power for when we are not connecting the ESP32 to a computer, as we only will do that for uploading the code, testing the components, and testing the whole circuit.

e. *Ultrasonic Sensor*
   i. This component sends out a high-frequency auditory signal, that then bounces off nearby obstacles and receives that signal back. It measures the time it takes to receive that signal to determine how close the obstacle is.

f. *Buzzer*
   i. There are two types of buzzers to choose from, active or passive, we chose a passive buzzer due to it's ability to make a wide verity of sounds. This way, when the buzzer receives the signal, it can make a sound at a pitch that becomes higher the closer the person gets to the obstacle the signal is bouncing off of.

g. *ERM Motor*
   i. ERM Motors function by having an unbalanced weight attached to a mechanical shaft. Our ERM vibration motor receives the auditory signal from the sensor and starts spinning unbalanced weights.

h. *Arduino IDE*
   i. Arduino IDE is a simple programming environment that works well with our ESP32, so we'll be writing our code with it. We'll be writing in C++ and will be using it's Serial Monitor to see what the components and

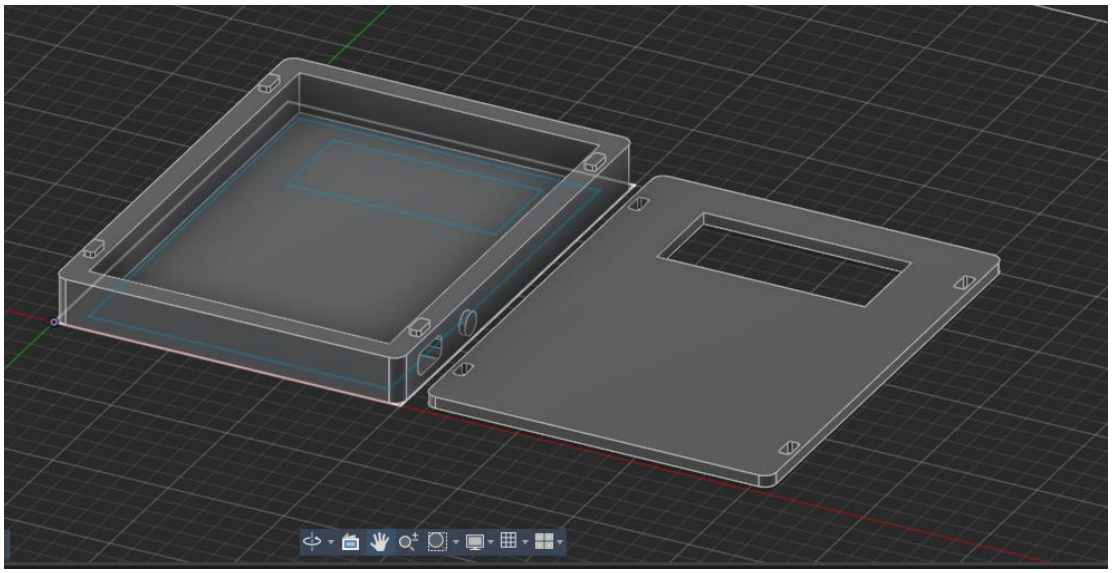
sensors are reading, we'll be able to see what our whole system is doing. It's like our output when writing a program.

V. Key Code Components (JG)

a. Many of the functions will use constants so that the software is easily modifiable. We want this to be open source. Many of the functions in this are for doing one task and one task only.
b. These are the key functions and what they should do:
    i. Setup Function
        1. This function will set up what pin on the ESP32 is associated with which component.
    ii. Loop Function
        1. This function will detail what the ESP32 will do while it is powered on. This will call for the sensors to read data and send alerts if necessary.
    iii. Process Sensor Data Function
        1. This will take in infromation from the Ultrasonic sensor in order to determine if an object is close enough to call for an alert. Another function is called for reading the distance and then providing it for checking whether an alert should be sent.
    iv. Send Alert Function
        1. This is a whole structure and function as an alert will have the type, and the distance associated with it if needed. This function is for making the alert and having it send a signal to the buzzer and the vibration motor to provide feedback to the user.
c. There may be more functions if they are needed.

VI. 3D Design (JP)

a. A 3d model of the design of case for our project was created using Autodesk fusion 360 and added to GitHub. The design was created using measurements of the hardware being used in the project, such as the breadboard, sensor, and buzzer. Due to the Parametric design of Fusion 360 if the dimensions of the can easily be changed if we decide to add or change any hardware. The case will be two separate parts, the base and the top. The pieces will connect together using a peg and hole design. The top has a window for the ultrasonic sensor.

b. 

Implementation

I. Connecting Hardware (YC)

a. Hardware Connection & Circuit Protection

i. Pin Mapping (5-Way Radar): All 5 ultrasonic sensors share a single TRIG pin (GPIO 5) to optimize pin usage. The 5 independent ECHO pins are mapped to GPIO 26, 25, 18, 33, and 32. The Passive Buzzer is assigned to GPIO 21.

ii. The Power Risk: A standard ESP32 GPIO pin can only safely output 12mA to 40mA. However, each 1030 ERM vibration motor requires over 120mA to start. Connecting motors directly to the ESP32 will instantly destroy the microcontroller's internal circuitry.

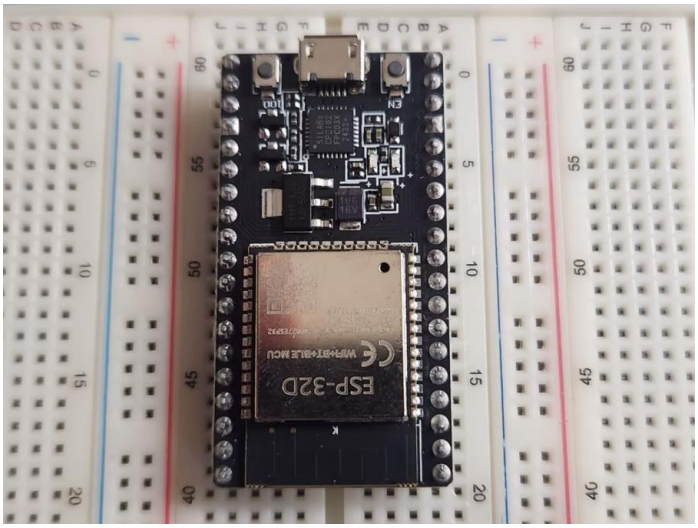
iii. Isolation Circuitry: To bridge this current gap and ensure electrical safety, an isolated protection loop is built for each motor branch:

iv. NPN Transistor (S8050/2N2222): Acts as an electronic switch. The ESP32 pin only outputs a safe ~1mA to the *Base*, while the motor safely draws its 120mA load directly from the power rail.

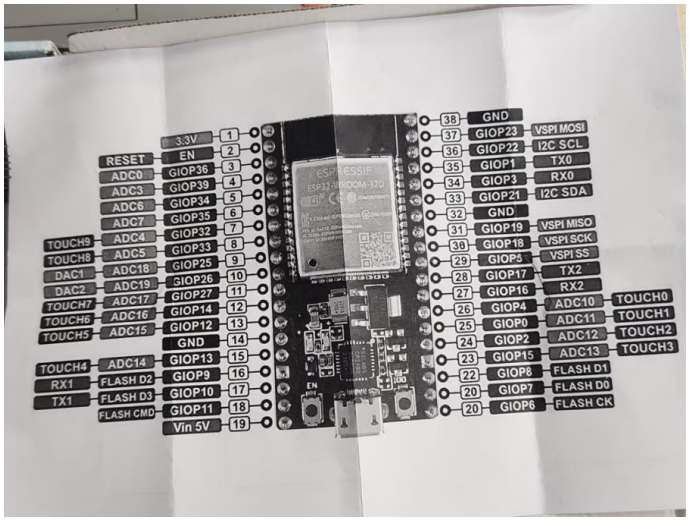
v. Flyback Diode (1N4148): Connected in reverse-parallel across the motor to suppress the high-voltage spike (inductive kickback) generated when the motor turns off.

vi. 1kΩ Resistor: Placed between the ESP32 pin and the transistor base to prevent a short-circuit.

b. Photos with Explanations

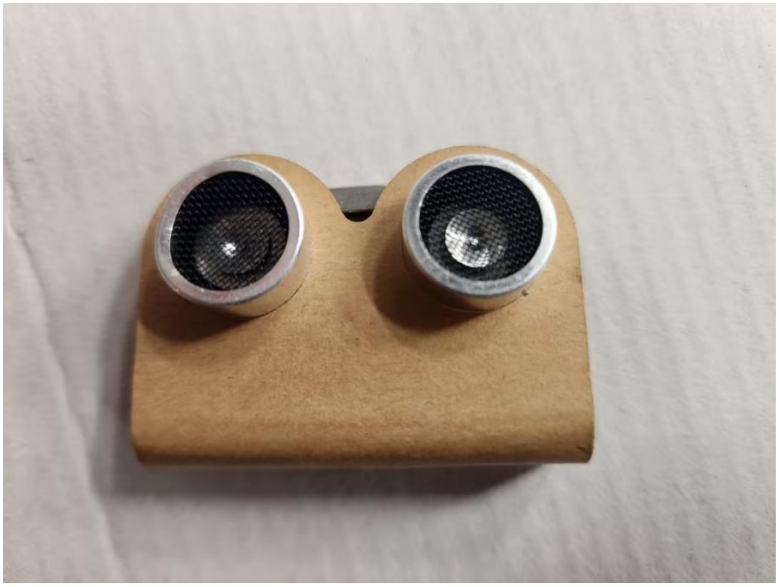


i. ESP32. The voltage is 5V (the internal working voltage of the chip is around 3.3V). The working current is 100 to 250mA. It can be powered by a power bank, and the maximum output current of the pins is 12 to 40mA.

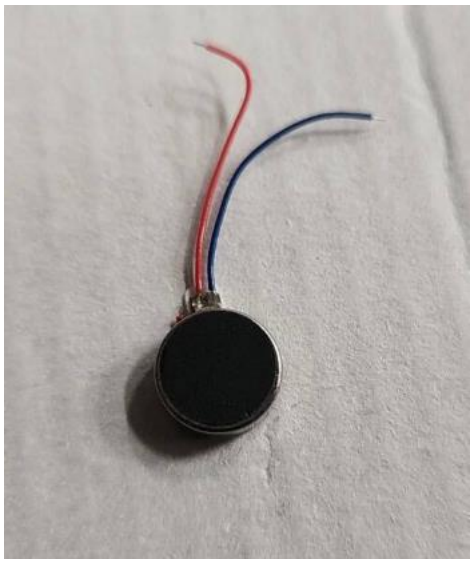




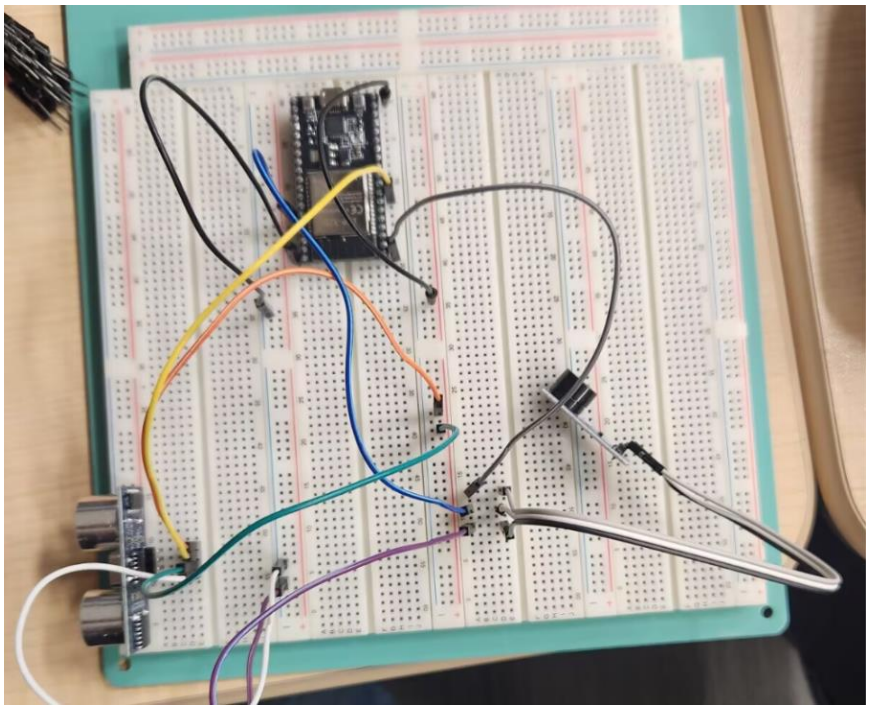
ii. This is a passive buzzer module. Its main function is to emit warning sounds. The operating voltage ranges from 3.3V to 5V, and the working current is approximately 30mA. It can be directly connected to the circuit.



iii. This is the HC-SR04 ultrasonic detector. Its working voltage is 5V and the working current is approximately 30mA. This can be directly connected to the ESP32 circuit board and does not require additional circuits for buffering and protection.



iv. This is the front and back sides of the motor. The working voltage of this motor is 3.3 volts DC, but the working current is 80 to 120 mA. To protect the circuit, components such as transistors need to be connected; otherwise, there is a risk of circuit breakdown.



II. Software Progress (JG)

a. Current progress (10/26/260 is focusing on making the main methods necessary for using the sensors and sending out alerts. Methods are made but need to be tested first.

III. Testing (JG)

a. To test out product we will use a set of different objects and use ourselves to recreate obstacles that the user may come in contact with. We will test the performance of the object detection and the alert system. We plan to focus on the key features to make a functioning product, then once the product does as intended, we can add more features to expand our scope.